

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0305	Course Title:	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	Total Marks:	100
CO2 Statement:	Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.		
Student Name:	G. Santosh Kumar	Reg. No.:	1A5U9020
Section / Batch:	2025	Date:	20/07/2026

Code of Conduct

I, G. Santosh Kumar (Name / Reg No)
certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the Student: G. Santosh Kumar

Instructions

- This test contains 5 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
GRAND TOTAL:			100 Marks

5	192511465	DHIVYADHARSAN M S	1. Identify if the given expressions are balanced or not. (i) $[9 * (k+8 - \{ 6 + e \} + 77) / 100]$, (ii) $((a*b) + [u/2] - d)$. 2. Given a stack with repeated elements, design an approach to compress it (value & frequency) while maintaining stack operations. Apply stack ADT modifications. Analyze trade-off between time and space
6	192521343	GOPIKA A	1. Two algorithms solve the same problem. Algorithm A uses recursion, while Algorithm B uses an explicit stack. Compare both approaches in terms of memory usage, execution flow, and risk of stack overflow. 2. A stack has a capacity of 5 elements. Explain what happens if the following operations are executed: Push(A), Push(B), Push(C), Push(D), Push(E), Push(F) Identify the error and suggest two possible solutions.
7	192524020	GULLAPALLI SANTOSH KUMAR	1. Insert the following elements 100, 200, 300, 400, 500 into the linked stack and pop the elements 100 and 200 from the linked stack. Show the linked structure before and after linked stack operations. 2. A circular queue has a size of 6. Execute the following operations: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60), Enqueue(70). Draw the queue after each operation and identify the values of the front and rear pointers.
8	192511397	H SARAVANA	1. Compare the performance of a linear queue and a circular queue when 100 enqueue and dequeue operations are performed alternately. 2. A queue implemented using an array report "Queue Full" even though some positions are empty. a. Analyze why this occurs. b. Explain how a circular queue solves this problem.
9	192525345	INTURI JASWANTH	1. Consider the singly linked list: $10 \rightarrow 20 \rightarrow 30 \rightarrow 40 \rightarrow 50$ Delete the node containing 30. Explain how the links change after deletion. 2. Given two sorted linked lists: L1: $1 \rightarrow 3 \rightarrow 5 \rightarrow 7$ L2: $2 \rightarrow 4 \rightarrow 6 \rightarrow 8$
10	192525211	JUJARU MEHER KOUSHIK	1. Convert the following infix expression into postfix notation using a stack. Show the contents of the stack after processing each symbol. $(A+B)*(C-D)/E$ 2. Convert the following infix expression into postfix form. Explain how operator precedence and associativity influence the conversion. $A+B*C-(D/E+F)*G$
11	192511070	K SUTHESIKA	1. Insert the following elements 150, 250, 350, 450, 550 into the double linked list and delete the elements 150 and 250 from the double linked list. Show the linked structure using a node diagram before and after delete operations.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

linked stack & pop the Elements 100 & 200 from the linked stack.

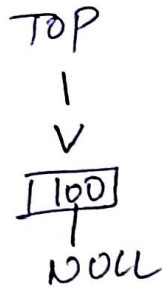
Q. Santosh Kumar

192529020

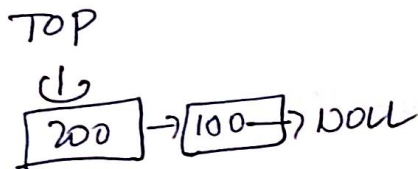
CSA0305

CO2-AT1 - Analytical

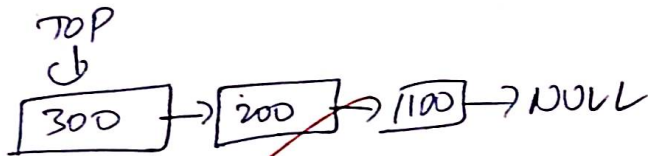
Step: 01 push 100



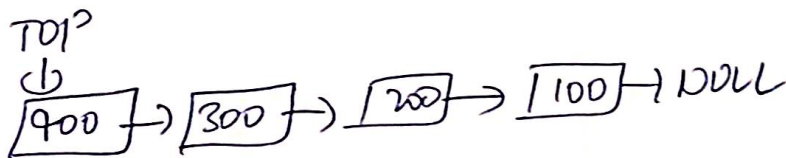
Step: 02 push 200



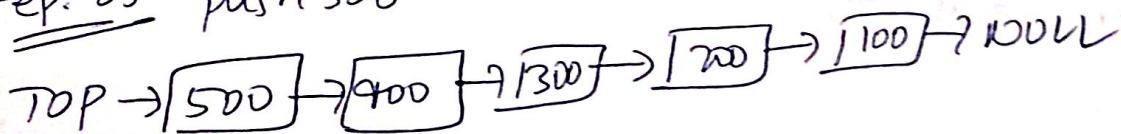
Step: 03 push 300



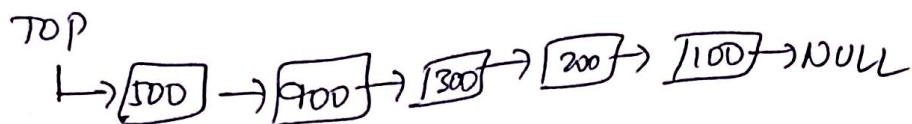
Step: 04 push 400



Step: 05 push 500



Linked state Before pop



pop operations

- first pop = 500
- second pop = 700

Linked stack after pop



Result

- Inserted Elements : 100, 200, 300, 700, 500
- popped Elements : 500, 700
- Remaining stack : 300 → 200 → 100

Q2. Circular Queue (Size = 6)

operations:-

- EnQueue (10)
- EnQueue (20)
- EnQueue (30)
- DeQueue()
- EnQueue (70)
- EnQueue (50)
- EnQueue (60)
- EnQueue (70)

Initial Queue

[-] [-] [-] [-] [-] [-]

front = -1

Rear = -1

EnQueue(10)

[10] [-] [-] [-] [-] [-]

~~front = 0~~

Rear = 0

EnQueue(20)

[10] [20] [-] [-] [-] [-]

front = 0

Rear = 1

EnQueue(30)

[10] [20] [30] [-] [-] [-]

DeQueue()

~~[-] [10] [30] [-] [-] [-]~~

~~front = 1~~

Rear = 3

EnQueue(40)

[-] [10] [30] [40] [-] [-]

front = 1

Rear = 3

EnQueue (50)

$[-][20][30][40][50][-]$

front = 1

Rear = 4

EnQueue (60)

$[-][20][30][40][50][60]$

front = 1

Rear = 5

EnQueue (70)

$[70][20][30][40][50][60]$

front = 1

Rear = 20

Final Circular Queue

